Viva script — Content Recommendation Feedback Dashboard

Viva script · Content Recommendation Feedback Dashboard · about 9 minutes

Read top to bottom. **SAY** is what comes out of your mouth. **SHOW** is what you click or point at while saying it. Square brackets are stage directions, not speech.

Total: about 9 minutes, plus questions.

Before anyone walks in

- ☐ Docker Desktop running; `docker ps --filter name=crfd` shows four containers.
- ☐ Tabs open in this order: dashboard (8080) · Jenkins job (8085) · build #14 · Docker Desktop · H2 console · GitHub repo.
- ☐ Dashboard shows events, not zeros. If the KPI cards read `0`, press **Generate traffic** once.
- ☐ Jenkins opens without a login (security is off on this demo controller).
- ☐ If you want build #14 on screen, open it now; do not start a new build unless you have four minutes.

1. Opening — 30 seconds

SAY:

> Good morning. My project is the Automated Content Recommendation Feedback Dashboard.
>
> The problem it solves is quiet failure. When a recommendation system gets worse, nothing crashes — the recommendations simply stop matching, and nobody notices until revenue drops.
>
> So the project does two things. It makes that degradation visible on a dashboard, and it wraps the product in a DevOps pipeline that makes a bad release impossible to ship silently.

SHOW: dashboard at `http://localhost:8080`.

2. What the product does — 60 seconds

SAY:

> It covers six capabilities. Recommendation events come in through an API. They are validated and classified, and the dashboard shows whether the pipeline is healthy: how many events arrived, the success rate, click-through rate and latency.

> I can search and filter the events, open one event and see its state, its audit trail and any alerts it raised, and I can work the alert queue — acknowledge and resolve exceptions. Users can also leave explicit feedback with a rating and a reason.
>
> Two decisions I want to point out. First, validation has two layers: a malformed request is rejected with a 400, but a well-formed event that cannot be used — say, one with no item — is accepted as *quarantined* and raises an alert. The system shows the exception instead of silently dropping traffic.
>
> Second, readiness is not the same as the service objective. Health returns 200 for UP and DEGRADED, because the process can still serve traffic; it returns 503 only when it is down. The SLO endpoint is the one that returns 503 when the success rate drops below 95 percent. An orchestrator should restart on the first; an operator should be paged on the second.

SHOW: scroll the KPI cards, click one event row for the drill-down, switch to the Exceptions tab.

3. The pipeline — 2 minutes

SHOW: `http://localhost:8085/job/crfd-dashboard-pipeline/` then build #14 → Stage View.

SAY:

> The pipeline is defined in a Jenkinsfile in the repository. Jenkins polls the develop branch, so a commit starts a build within about five minutes.
>
> There are ten stages.
>
> It checks out the code and works out the version, the commit hash and the image tag for this build. Then it runs the Maven build, which executes the unit and API tests and collects coverage. Then it packages the runnable JAR.
>
> Stage four builds the Docker image, tagged with the build number, so every build produces an immutable image.
>
> Stage five is the one that matters most. Selenium starts the image as a throwaway container, launches real Chrome, and walks five critical user journeys: the dashboard renders, an event can be entered and found, search and filtering work, the drill-down shows the audit trail, and an operator can acknowledge an alert.

> If any of those fail, stage eight never runs. That is the quality gate: a red build cannot replace the running application. Build number seven on this job is red for exactly that reason — Selenium failed, the deploy was skipped, and the old container kept serving.
>
> Stages six and seven publish the test reports and, optionally, push the image to a registry. Stage eight removes the old container and starts the new image against the same data volume. Stage nine waits for the health endpoint to return 200, then runs smoke checks through the API. Stage ten is the optional Ansible provisioning step.
>
> Every stage's output is archived on the build page — the JAR, the health response, the smoke responses, the image details and the container log.

SHOW: console tail with `Deployed crfd:v3.3.5-14`; Test Result (37 tests, 0 failures); Coverage (71% lines); Artifacts list; then build #7 in red.

4. How Jenkins is used — 90 seconds

SAY:

> Jenkins is itself a container. It is given the host's Docker socket, so when the pipeline runs Docker it is asking the host daemon to build and run containers. That means the application container is a sibling of Jenkins, not a child.
>
> That one fact explains the most common mistake in this setup: inside the Jenkins container, localhost is Jenkins itself. So the pipeline addresses the application as `crfd-app` or `crfd-live` on the shared Docker network, and Selenium does the same.
>
> The controller is configured as code. A compose file defines it, a plugin list defines what it can do, and a Configuration-as-Code file defines the tools, the credentials and a seed job. The seed job creates this pipeline from the Jenkinsfile in the repository.
>
> That means nothing about the CI server is hand-clicked. If I lose the container, I bring the same controller back with one command.
>
> The job is parameterised: environment, port, whether to run Selenium, whether to publish the image, whether to deploy. And two environment settings — the low-confidence threshold and the critical latency limit — let the same image behave differently per environment without rebuilding.

> On this local demo the login is switched off deliberately, because I am demonstrating on localhost. The configuration file documents exactly how to switch it back on.

5. How Docker is used — 90 seconds

SHOW: Docker Desktop → Containers, filter `crfd`.

SAY:

> The image is built in two stages. The first stage uses Maven and JDK 21 to build the JAR, and none of that ships. The second stage is a JRE-only Alpine image, about four hundred megabytes. It runs as a non-root user, it declares a volume for the data, and it carries its own health check, so Docker itself knows whether the container is healthy.
>
> There is no database container. The MVP uses H2 in file mode, stored on the `crfd-data` volume. That is a deliberate choice for a single-node MVP: no database server to run or patch, and the data still survives the container being deleted. The MySQL path is already written for when more than one instance has to share the data.
>
> Every build produces a new tag — thirteen, fourteen, and so on. That tag list is the rollback mechanism. Rolling back is starting the previous tag against the same volume.
>
> Four containers are running: the application, the Jenkins controller, a standalone Chrome for the Selenium gate, and a local registry. They are all on one bridge network, which is how they find each other by name.

SHOW: `crfd-app` → Inspect → Mounts (`crfd-data` → `/app/crfd-data`); Files → `/app/crfd-data/crfd.mv.db`; Images tab with the tag list.

6. How we executed it — 2 minutes

SHOW: build #14, then build #13, then the H2 console.

SAY:

> I proved the product locally first. The Maven build with thirty-two unit and API tests, then Docker Compose for the application and Nginx, then Selenium locally and against the Selenium container, then the Ansible playbook through five steps: provision a clean node, run it a second time to prove idempotency, deploy the release, check health, and roll back.

> Then I stood up Jenkins and ran the pipeline repeatedly. The first completely green run was build eight, and nine through thirteen followed.
>
> Build thirteen is honest evidence of how real this was. It succeeded, but it took thirty-seven minutes, because the Docker stage spent two thousand one hundred seconds waiting on a slow Alpine package mirror. I fixed it by caching the image layer instead of hiding the problem.
>
> Today's build, fourteen, is the same pipeline on a warm cache: three minutes forty-five in total. Docker build sixty-four seconds, Selenium seventy-seven, deploy under three seconds, health and smoke eighteen seconds.
>
> After the deploy I verified it three ways. The health endpoint returns 200 with status UP and a success rate of 95.6 percent. The smoke test ingests an event, reads it back and checks the summary. And the H2 console shows the real rows in the database: six hundred and thirty-one processed, thirty-five failed, six quarantined, five still processing.
>
> One thing I will point out rather than hide. Immediately after a build the dashboard can read DEGRADED, because the Selenium journey deliberately posts one invalid event to prove quarantine. That single rejection nudges the twenty-four-hour success rate under ninety-five percent. Readiness stays 200 — the application is fine. The SLO endpoint goes 503, because the business objective is breached. That is exactly the distinction I designed, and it is visible in the build.

7. Closing — 20 seconds

SAY:

> So the end result is a product that shows recommendation quality, and a pipeline that builds, tests, containerises, gates, deploys and verifies it on every commit.
>
> What I would do next is at the scale-out end: MySQL for shared state, several application replicas behind Nginx, images pushed to a real registry, then Kubernetes for scheduling and rolling updates — keeping the same Jenkins gates in front of it.
>
> Thank you. I am happy to take questions.

Numbers to memorise

Number	Meaning

37	tests per pipeline run — 12 unit, 20 API, 5 Selenium
32	tests before the pipeline adds Selenium
71% / 45%	line / branch coverage
10	pipeline stages
14	last build, SUCCESS, 225 s
63.9 s	Docker stage on a warm cache (versus 2 104 s in #13)
37 min	build #13, the slow one
401 MB	runtime image
4	containers: app, Jenkins, Selenium, registry
3	volumes: data, Jenkins home, registry data
8080 / 8085	app / Jenkins
95.6%	current 24-hour success rate
681	events currently in the database

One-line answers for questions

- **Why H2?** Single-node MVP, zero database operations, data on a volume so the container stays disposable; MySQL profile is the scale-out path.
- **Why not Kubernetes?** Out of scope for a single-node MVP; Compose plus the Jenkins deploy gives repeatable deployments. Listed as future work.
- **How do you know a deploy worked?** Healthy container, health endpoint 200, eleven smoke checks, all archived as build artifacts.
- **Rollback?** Start the previous immutable image tag against the same volume; the Ansible playbook also has a rollback step.
- **What stops a bad release?** Selenium runs before deploy; failure skips the deploy stage. Build #7 is the proof.
- **Idempotency?** Ansible second run reports no changes; deploy removes and recreates the same way every time; JCasC rebuilds the same controller.
- **Readiness versus SLO?** Readiness is 200 for UP and DEGRADED and 503 only when DOWN; the SLO endpoint is 503 when the success rate is under 95%.
- **Secrets?** Jenkins credential store, injected at build time; the repo holds placeholders only.
- **Why is Jenkins in Docker?** So the CI server is reproducible from source control in one command.
- **Hardest part?** Docker-outside-of-Docker networking, and Jenkins Configuration-as-Code strictness — both documented with their fixes.
- **What is not covered?** Dashboard authentication, multi-tenancy, high availability, TLS, and a coverage gate.
- **Scale-out?** MySQL, replicas behind Nginx, real registry, then Kubernetes.

If something goes wrong mid-demo

Symptom	Quick fix
KPI cards show 0	press **Generate traffic** once
Badge says DEGRADED	explain readiness vs SLO; it is the Selenium rejection

Jenkins asks for a login	re-apply `useSecurity=false` and restart the container
"Build with parameters" fails	run one plain **Build now** first
Port 8080 in use	deploy with `APP_PORT=18080`
App container missing	re-run the deploy stage, or run the newest `crfd:v3.3.5-*` tag

Commands worth having open

```
docker ps --filter name=crfd
docker logs -f crfd-app
docker inspect crfd-app --format '{{json .Mounts}}'
curl -s http://localhost:8080/api/v1/health
curl -s -o NUL -w "%{http_code}" http://localhost:8080/api/v1/health/slo
docker exec crfd-app ls -lh /app/crfd-data
./mvnw -B verify
./mvnw -B -P selenium verify -Dselenium.baseUrl=http://localhost:8080
docker build -t crfd:demo .
docker run -d --name crfd-try -p 18080:8080 -v crfd-data:/app/crfd-data crfd:demo
```

H2 console: `jdbc:h2:file:/app/crfd-data/crfd`, user `sa`, empty password.

```
SELECT status, COUNT(*) FROM recommendation_event GROUP BY status;
SELECT code, severity, status, COUNT(*) FROM alert GROUP BY code, severity, status;
```